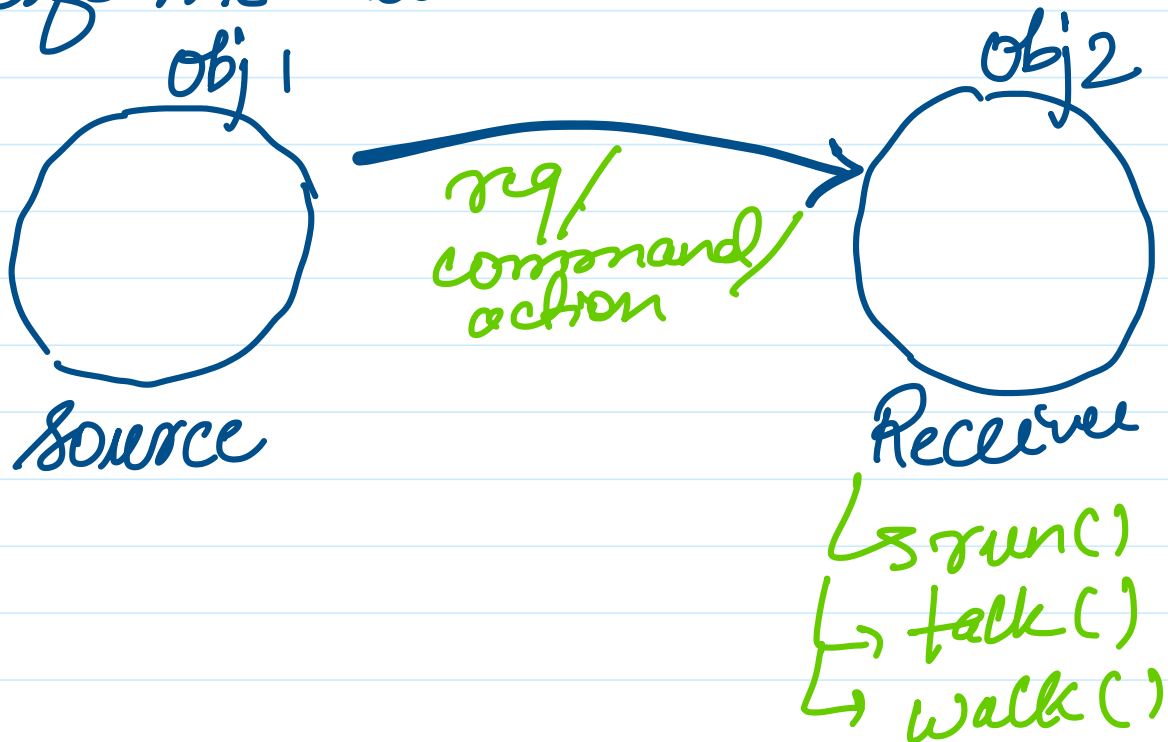


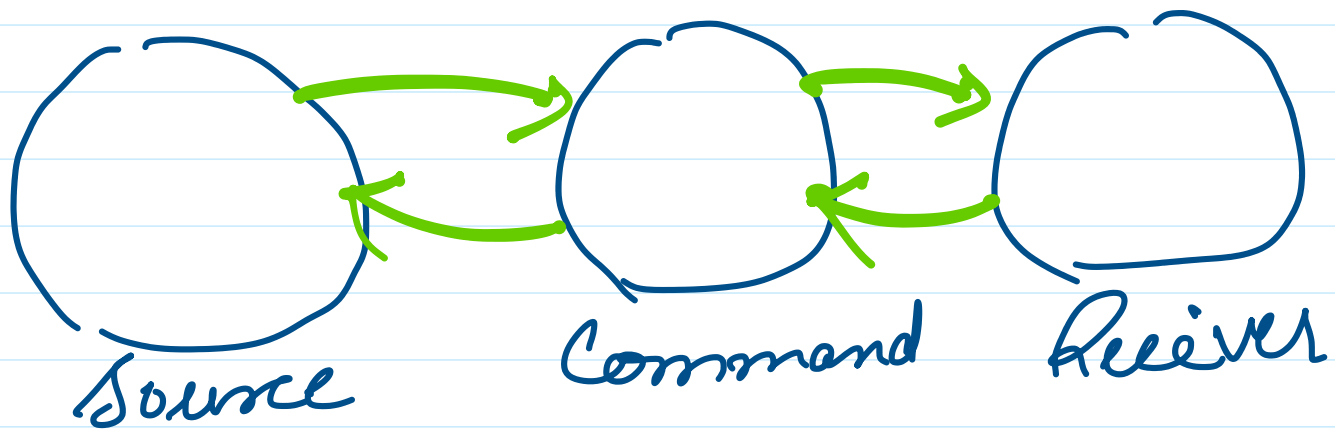
Command Pattern

I want source to send request/command/action to receiver & receiver performs it



We can treat req / command / action also an object so,

Now source will interact with command object and it will make receiver work and send result back to source

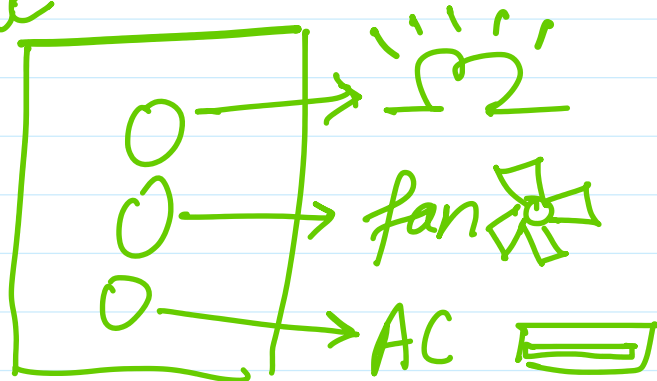


Why you created Command as object
So, that source & Receiver becomes
loosely coupled

Example

Smart Home Automation

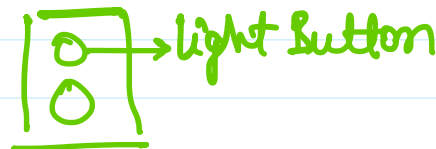
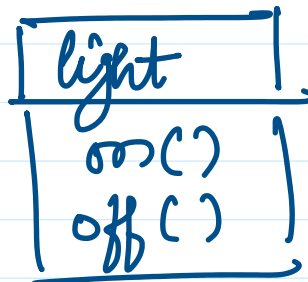
Remote



We have Button by which we are controlling home appliance

If we don't know Command Pattern we have done this

```
class Remote {
    light light;
    PressLightButton() {
        light → on()
    }
}
```



Now we have tightly coupled the Remote & light

Suppose in future we want light Button to do something else like controlling heater
 i.e. all over the Remote class

controlling meter

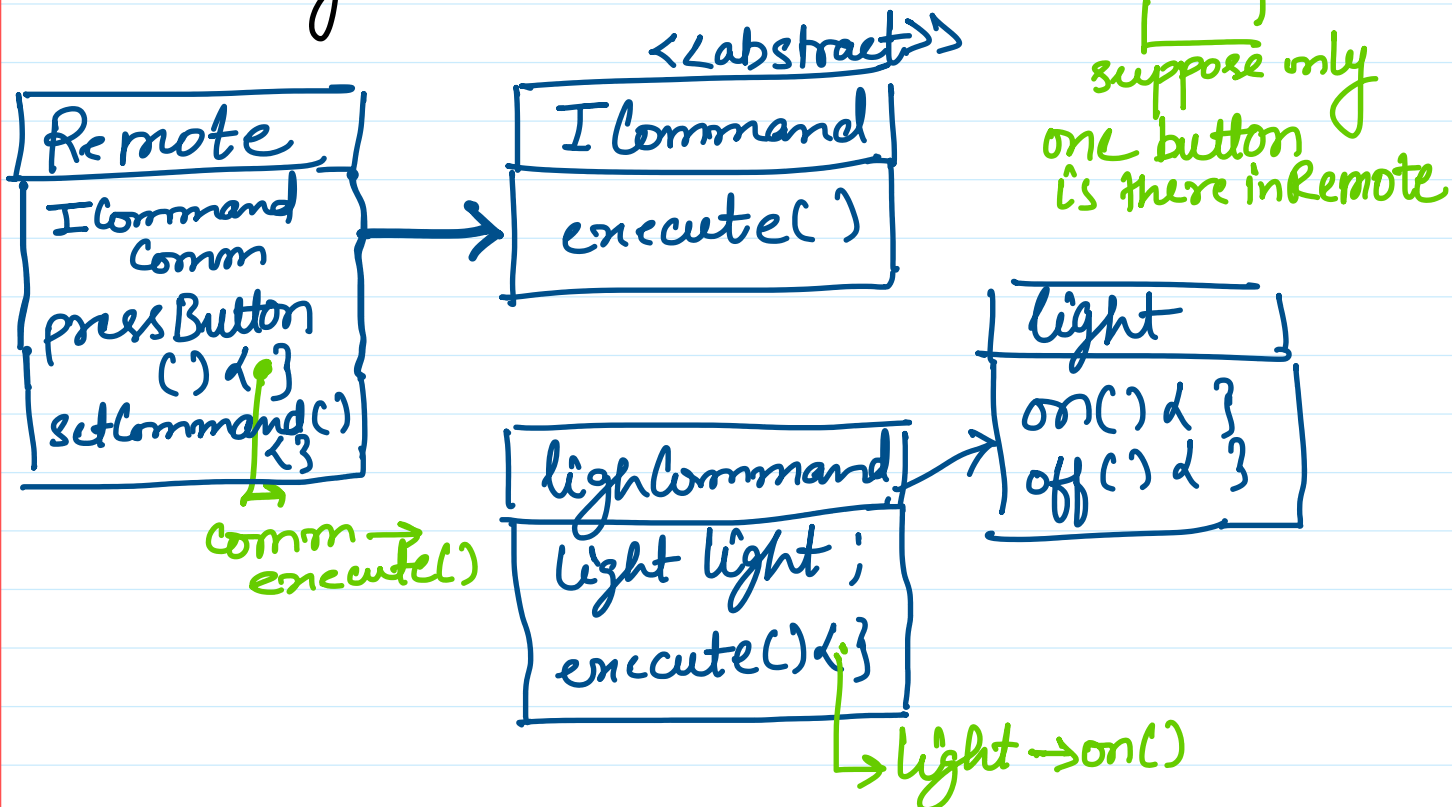
then we will open the Remote class
and change the code

↳ this is wrong principal

OCP X

So, our Remote should not be directly
interact with appliance. Instead
it should link with Command and
Remote should tell Command to do
the work and Command should know
which appliance to call.

UML Diagram :-



In 'Comm' we will set any Command
like 'light Command' by using
`setCommand()` method

And we will simply call `pressButton()` and light will get 'on'. Remote doesn't know what happens by pressing the Button

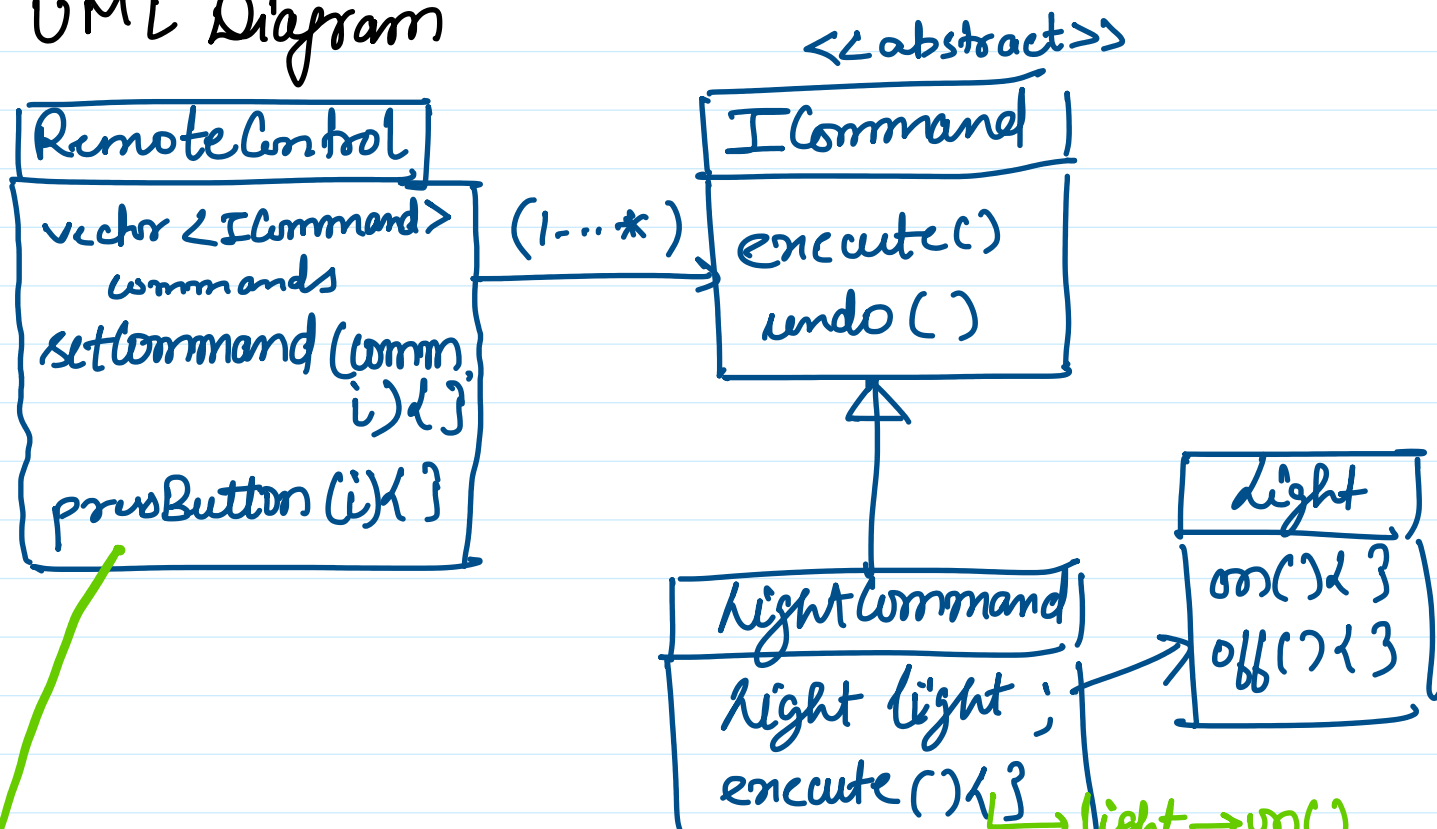
It totally depends upon what we have set in 'Comm' variable.

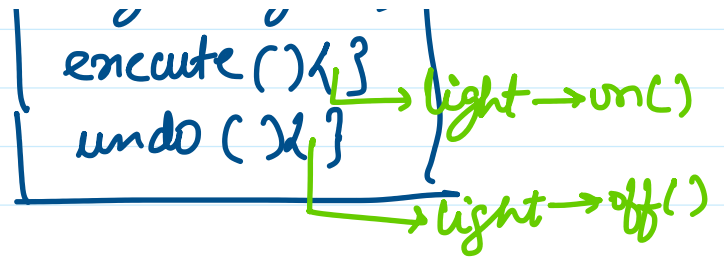
In future if you want to change the functionality of that Button to switch on the fan. Then you can simply set the 'fanCommand' in 'comm' variable

Now Suppose we have multiple Buttons on Remote :



UML Diagram





So here we took the list of ICommand
Now setCommand take Index i to set
Command on particular index.

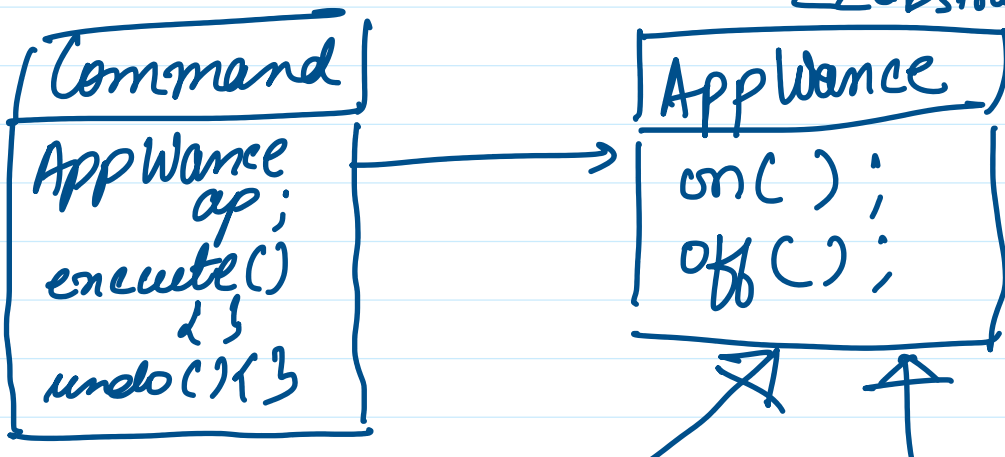
Same pressButton() also take index i
to execute command at particular index.

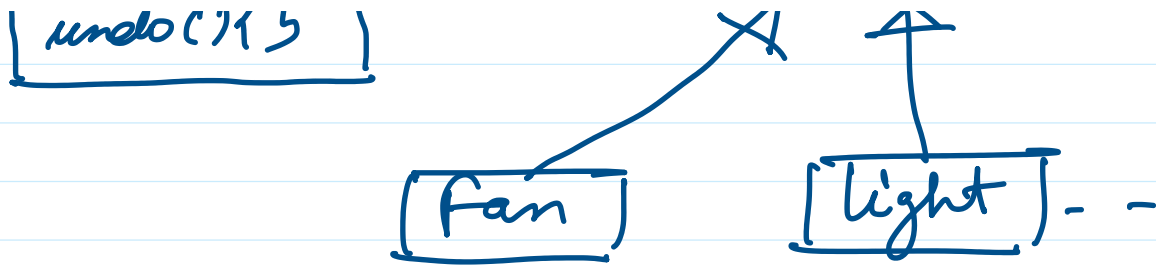
→ We can have boolean pressed, to
know wheather the button was
pressed or not so that we can
switch b/w on & off. (execute() & Undo())

Code → see Notes

Note :- In this light Command
has-a relationship with light
Concrete Class $\xrightarrow{\text{has-a}}$ Concrete Class

We could have done :- <<Abstract>>





We didn't do this b/c - There can be many appliance which may not have `on()` & `off()` feature... we might have narrow down our Child Class

LSP X

Use Cases :-

— Text Editor , Photoshop

↳ In this we need undo() feature Ctrl+Z

Any application where undo() feature is required. on some action like Bold, Italics
Command

— Keyboard Shortcut—

You set shortcut on keyboard to give command to perform some action.

You can change the shortcut to perform some other task as well.

We can use this Pattern where we are setting some command and in future we can dynamically change it.

Definition :-

1. ... 1. to ... 1. ... 1. ...

Encapsulate a Request as an Object, thereby letting you parameterizing Clients with different request, queue or log request & support undoable operations.